

FIAP

PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE · 2º SEMESTRE

Pipeline RAG completo

Aula 06/14 — Módulo 2: RAG · load → split → embed → retrieve → generate



Prof. Jorge Luiz Gomes



1h40min



PyMuPDF · RecursiveCharacterTextSplitter



Andaime 50%

Agenda da Aula

- 1** O que é RAG — por que o LLM não precisa saber tudo se consegue buscar 15 min
- 2** Etapas 1–3 — carregar PDFs (PyMuPDF), dividir em chunks, gerar embeddings 20 min
- 3** Etapas 4–6 — armazenar no ChromaDB, recuperar top-k, gerar com citação de fonte 15 min
- 4** Chain RAG em LCEL — retriever | prompt | llm | parser em 50 linhas 15 min
- 5** 🚀 Demo — RAG sobre 3 PDFs: modelo sem RAG (alucina) vs. com RAG (cita fonte) 15 min
- 6** 📄 Lab — RAG sobre PDFs do domínio do grupo (50% lacunas) 20 min

🎯 OBJETIVO DA AULA

Construir um pipeline RAG completo funcional em ~50 linhas. O LLM responde sobre documentos reais que nunca estiveram no treinamento — com citação de página e trecho. Essa é a fundação do CKP02.

Glossário

RAG /ér.ei.dji/

Retrieval-Augmented Generation — técnica que busca trechos relevantes em uma base de documentos e injeta no contexto do LLM antes de gerar a resposta.

Analogia: prova de consulta em vez de prova de memorização — o modelo pode "abrir o livro" antes de responder.

Chunk / Chunking

Divisão de um documento longo em pedaços menores antes de gerar embeddings — melhora a precisão da busca por similaridade.

chunk_overlap

Quantidade de caracteres repetidos entre um chunk e o próximo, para não perder uma informação que ficaria cortada exatamente na fronteira entre dois chunks.

Grounding

Ancorar a resposta do modelo em uma fonte verificável — o prompt instrui o LLM a responder somente com base no contexto recuperado e citar a fonte.

Analogia: responder citando a página do livro, em vez de "confiar na memória".

PyMuPDF

Biblioteca Python (módulo `fitz`) para extrair texto de PDFs preservando número de página — essencial para citar a fonte da resposta.

Retriever (top-k)

Componente que recebe uma pergunta, converte em embedding e devolve os k chunks mais similares do vector store — a peça que conecta o ChromaDB à chain RAG.

RunnablePassthrough

Runnable do LangChain que encaminha o input sem transformação — usado quando um valor (como a pergunta original) deve chegar intacto a uma etapa seguinte da chain.

RunnableLambda

Runnable que envolve qualquer função Python, tornando-a composável com o operador `|` dentro de uma chain LCEL.

01

O que é RAG

Retrieval-Augmented Generation — o LLM não precisa memorizar tudo. Ele busca o que precisa, injeta no contexto e gera a resposta fundamentada na fonte.

O problema que RAG resolve

LLMs têm dois problemas fundamentais para uso em produção com documentos específicos:

❌ PROBLEMA 1 — ALUCINAÇÃO

Pergunta: "Qual é a cláusula 7.3 do contrato da empresa X?"

Modelo sem RAG: "A cláusula 7.3 estabelece que o prazo de entrega é de 30 dias..."

O modelo inventou — o contrato não estava no treinamento.

❌ PROBLEMA 2 — DESATUALIZAÇÃO

O treinamento tem data de corte. Regulamentos, manuais, preços, documentos internos — tudo que surgiu depois do corte é invisível para o modelo.

✅ SOLUÇÃO — RAG

Pergunta: "Qual é a cláusula 7.3 do contrato?"

RAG busca no ChromaDB → recupera o trecho do contrato → injeta no contexto.

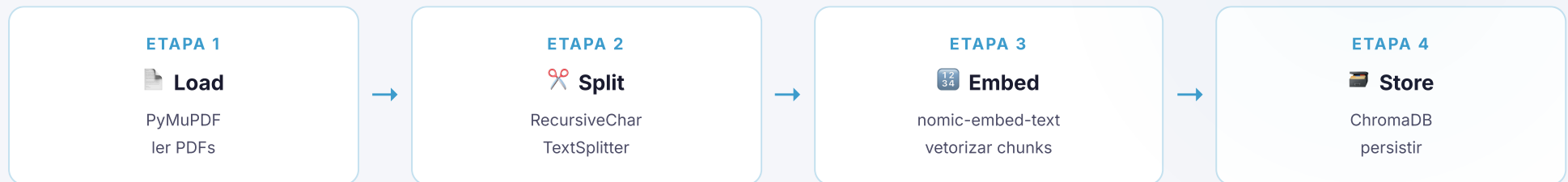
Modelo com RAG: "Conforme o documento fornecido, a cláusula 7.3 (página 12) estabelece que..."

Resposta fundamentada na fonte real, com citação.

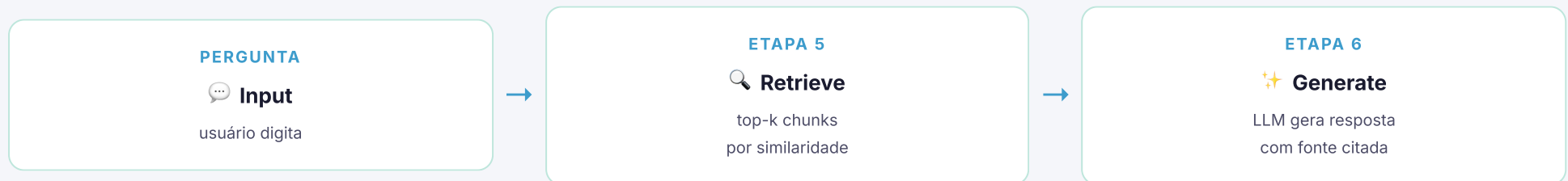
As 6 etapas do pipeline RAG

O pipeline tem duas fases: **indexação** (feita uma vez) e **inferência** (a cada pergunta do usuário):

FASE 1 — INDEXAÇÃO (EXECUTA UMA VEZ)



FASE 2 — INFERÊNCIA (A CADA PERGUNTA)



💡 RAG = MEMÓRIA EXTERNA SOB DEMANDA

O contexto do LLM é o "RAM" — rápido mas finito. O ChromaDB é o "HD" — ilimitado mas precisa de busca. RAG recupera só o trecho relevante do HD e coloca na RAM apenas quando necessário. É por isso que funciona com documentos maiores que a janela de contexto.

02

Etapas 1–3

Load · Split · Embed

Carregar PDFs com PyMuPDF, dividir em chunks com

RecursiveCharacterTextSplitter, vetorizar com nomic-embed-text. A fase de indexação.

Etapa 1 — carregar PDFs com PyMuPDF

PyMuPDF (alias `fitz`) é a biblioteca mais robusta para extração de texto de PDFs — mantém números de página e funciona com PDFs com layout complexo:

```
ETAPA1_LOAD.PY

!pip install langchain langchain-community pymupdf langchain-ollama chromadb -q

from langchain_community.document_loaders import PyMuPDFLoader
from pathlib import Path

# Fazer upload do PDF no Colab
from google.colab import files
uploaded = files.upload() # abre o seletor de arquivos
pdf_path = list(uploaded.keys())[0]

# Carregar — cada página vira um Document
loader = PyMuPDFLoader(pdf_path)
paginas = loader.load()

print(f"Páginas carregadas: {len(paginas)}")
print(f"Metadados da pág. 1: {paginas[0].metadata}")
# → {'source': 'manual.pdf', 'page': 0, 'total_pages': 24, ...}

print(f"Trecho da pág. 1:\n{paginas[0].page_content[:300]}")

# Carregar múltiplos PDFs de uma vez
pdfs = ["manual_1.pdf", "manual_2.pdf", "regulamento.pdf"]
todas_paginas = []
for pdf in pdfs:
    todas_paginas.extend(PyMuPDFLoader(pdf).load())
print(f"Total de páginas: {len(todas_paginas)}")
```

✓ POR QUE PYMUPDF E NÃO PYPDF2?

PyMuPDF mantém metadados de página numerada automaticamente (`metadata["page"]`), lida melhor com PDFs escaneados e layouts de duas colunas, e é 3–5× mais rápido. Para citar a fonte da resposta ("página 12"), o metadado de página é essencial.

Etapa 2 — dividir em chunks com RecursiveCharacterTextSplitter

Páginas inteiras são grandes demais para um embedding preciso. Dividir em **chunks** menores melhora a precisão da recuperação:

```
ETAPA2_SPLIT.PY

from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,      # ~600-1000 chars por chunk (boa prática para RAG)
    chunk_overlap=100,   # sobreposição para não quebrar frases no limite
    separators=["\n\n", "\n", ". ", " ", ""], # ordem de preferência
)

# Dividir as páginas em chunks menores
chunks = splitter.split_documents(paginas)

print(f"Páginas originais: {len(paginas)}")
print(f"Chunks gerados: {len(chunks)}")
print(f"Exemplo de chunk:\n{chunks[0].page_content}")
print(f"Metadados: {chunks[0].metadata}")
# → {'source': 'manual.pdf', 'page': 0} — página preservada!

# Ver distribuição de tamanhos dos chunks
tamanhos = [len(c.page_content) for c in chunks]
import statistics
print(f"Tamanho médio: {statistics.mean(tamanhos):.0f} chars")
print(f"Tamanho mín/máx: {min(tamanhos)} / {max(tamanhos)} chars")
```

⚙️ POR QUE CHUNK_SIZE=800 E CHUNK_OVERLAP=100?

800 chars (~150–200 palavras) é o sweet spot para o nomic-embed-text — captura contexto suficiente sem diluir o significado. Overlap de 100 chars evita que uma informação no final de um chunk e início do próximo seja perdida na busca. Na Aula 07 (RAG avançado) você vai experimentar outros tamanhos e medir o impacto no RAGAS.

Etapas 3 e 4 — embed e store no ChromaDB

Vetorizar os chunks e armazenar no ChromaDB em uma única chamada com `Chroma.from_documents()` :

ETAPA3_4_EMBED_STORE.PY

```
from langchain_community.vectorstores import Chroma
from langchain_ollama import OllamaEmbeddings
import os
from google.colab import userdata

os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")

embeddings = OllamaEmbeddings(model="nomic-embed-text")

# Etapas 3 + 4 em uma linha: embed todos os chunks e salva no ChromaDB
db = Chroma.from_documents(
    documents=chunks,          # lista de chunks (Document)
    embedding=embeddings,      # modelo de embedding
    persist_directory="/content/rag_ckp02", # salva no disco
)
print(f"Chunks indexados: {db._collection.count()}")

# Retriever: interface de busca para a chain LCEL
retriever = db.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3}, # recuperar os 3 chunks mais similares
)

# Testar o retriever isolado
docs_recuperados = retriever.invoke("qual é o prazo de entrega?")
for d in docs_recuperados:
    pg = d.metadata.get("page", "?")
    print(f"Pág. {pg}: {d.page_content[:120]}...")
```

03

Chain RAG em LCEL

`retriever | prompt | llm | parser` — conectar as peças
em ~10 linhas.

O prompt de RAG — grounding e citação de fonte

O prompt de um sistema RAG tem uma responsabilidade extra: instruir o modelo a usar **somente** o contexto fornecido e citar a fonte:

```
PROMPT_RAG.PY

from langchain_core.prompts import ChatPromptTemplate

PROMPT_RAG = """<persona>
Você é um assistente especializado em responder perguntas
com base EXCLUSIVAMENTE nos documentos fornecidos.
</persona>

<instrucoes>
- Responda SOMENTE com informações do contexto abaixo.
- Sempre cite a fonte: (fonte: {nome_doc}, página X).
- Se a resposta não estiver no contexto, diga:
  "Não encontrei essa informação nos documentos fornecidos."
- Nunca invente, extrapole ou use conhecimento externo.
</instrucoes>

<contexto>
{contexto}
</contexto>

<pergunta>
{pergunta}
</pergunta>"""

prompt = ChatPromptTemplate.from_template(PROMPT_RAG)
```

🌟 GROUNDING — A PROPRIEDADE MAIS IMPORTANTE DO RAG

Grounding é ancorar a resposta em uma fonte verificável. O prompt acima tem três mecanismos de grounding: (1) "EXCLUSIVAMENTE nos documentos"; (2) citação de fonte obrigatória; (3) resposta padrão quando a informação não está no contexto — em vez de alucinar. Sem grounding explícito no prompt, o modelo ainda pode alucinar mesmo com RAG.

Chain RAG completa — ~50 linhas, pipeline funcional

CHAIN_RAG_COMPLETA.PY

```
from langchain_ollama import ChatOllama
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough, RunnableLambda

llm = ChatOllama(model="qwen3.6:27b", temperature=0) # temp=0 para RAG (menos variação)

def formatar_contexto(docs) -> str:
    """Formata os docs recuperados com número de página para citação."""
    partes = []
    for i, doc in enumerate(docs, 1):
        fonte = doc.metadata.get("source", "doc")
        pg = doc.metadata.get("page", "?")
        partes.append(f"[Trecho {i} — {fonte}, pág. {pg+1}]\n{doc.page_content}")
    return "\n\n---\n\n".join(partes)

# Chain LCEL completa — a magic pipe
chain_rag = (
    {
        "contexto": retriever | RunnableLambda(formatar_contexto),
        "pergunta": RunnablePassthrough(), # pergunta passa direto
        "nome_doc": RunnableLambda(lambda _: pdf_path),
    }
    | prompt
    | llm
    | StrOutputParser()
)

# Usar a chain
resposta = chain_rag.invoke("Qual é o prazo de entrega do produto?")
print(resposta)

# → "Conforme o documento manual.pdf, página 8, o prazo de entrega é..."
```

05

Lab — Notebook Aluno

50% de lacunas — pipeline RAG completo sobre os PDFs do domínio do grupo.

Ao final: chain funcionando, respondendo com citação de página.

Notebook Aluno — 50% de lacunas

AULA_06_2SEM_ALUNO.IPYNB

```
!pip install langchain langchain-community langchain-ollama pymupdf chromadb langchain-text-splitters -q
```

```
from langchain_community.document_loaders import PyMuPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_ollama import OllamaEmbeddings, ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough, RunnableLambda
import os
from google.colab import userdata, files
```

```
os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")
```

```
# 🚩 LACUNA 1: faça upload e carregue os PDFs do domínio
uploaded = files.upload()
pdf_paths = [_____] # lista de caminhos dos PDFs enviados
paginas = []
for p in pdf_paths:
    paginas.extend(PyMuPDFLoader(____).load())
```

```
# 🚩 LACUNA 2: crie o splitter e divida as páginas em chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=____, # 800 é um bom ponto de partida
    chunk_overlap=____, # 100 para não perder contexto nas bordas
)
chunks = splitter.split_documents(____)
print(f"Chunks: {len(chunks)}")
```

```
# 🚩 LACUNA 3: crie o vector store com nomic-embed-text
embeddings = OllamaEmbeddings(model=____)
db = Chroma.from_documents(____, embedding=____, persist_directory="/content/ckp02")
retriever = db.as_retriever(search_kwargs={"k":3})
```

```
# 🚩 LACUNA 4: monte e invoque a chain RAG com grounding e citação
chain_rag = (
    {"contexto": retriever | RunnableLambda(formatar_contexto),
     "pergunta": _____,
     "nome_doc": RunnableLambda(lambda _: "PDFs do domínio")
    | prompt | ChatOllama(model="qwen3.6:27b", temperature=0) | StrOutputParser()
)
print(chain_rag.invoke(____)) # sua pergunta sobre o domínio
```

Decisões de chunking — o que afeta a qualidade do RAG

O tamanho e a estratégia de chunking é uma das principais alavancas de qualidade de um sistema RAG:

PARÂMETRO	VALOR PEQUENO	VALOR GRANDE	RECOMENDAÇÃO
chunk_size	200–400 chars <i>Precisão alta, contexto pobre</i>	1500–2000 chars <i>Contexto rico, precisão diluída</i>	600–1000 chars <i>sweet spot para nomic</i>
chunk_overlap	0 (sem overlap) <i>Pode perder informação na borda</i>	50%+ do chunk <i>Redundância excessiva, mais custo</i>	10–15% do chunk_size <i>~100 para chunk de 800</i>
separators	[""] — split por char <i>Quebra no meio de palavras</i>	["\n\n"] só <i>Chunks muito irregulares</i>	["\n\n", "\n", ". ", " ", ""] <i>hierárquico — parágrafo primeiro</i>

📖 FUNDAMENTAÇÃO — POR QUE A ORDEM DOS SEPARATORS IMPORTA

Polzer (*RAG with Python Cookbook*, O'Reilly, 2026) explica o mecanismo por trás do `RecursiveCharacterTextSplitter` : ele tenta os separadores da lista na ordem, do mais estrutural para o mais granular — primeiro quebra em parágrafo (`\n\n`), só recorre a linha, depois frase, e usa espaço ou caractere como último recurso, quando o chunk ainda excede o tamanho máximo. É por isso que a lista recomendada nesta tabela é hierárquica e não arbitrária: cada separador é uma tentativa de preservar a estrutura que um humano usaria para organizar o texto, antes de partir para uma quebra puramente mecânica.

➡️ AULA 07 — RAG AVANÇADO

Na Aula 07 você vai experimentar diferentes valores de `chunk_size` e medir o impacto com RAGAS (faithfulness e answer relevancy). Guarde os resultados da chain de hoje como baseline para comparação.

Pipeline completo em bloco único (~50 linhas)

Visão consolidada do pipeline inteiro para referência rápida:

RAG_PIPELINE_COMPLETO.PY

```
# — 1. SETUP —————
import os
from google.colab import userdata
os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")

# — 2. INDEXAÇÃO (uma vez) —————
paginas = PyMuPDFLoader("documento.pdf").load()
chunks = RecursiveCharacterTextSplitter(800, 100).split_documents(paginas)
db = Chroma.from_documents(chunks, OllamaEmbeddings(model="nomic-embed-text"),
                           persist_directory="/content/rag")
retriever = db.as_retriever(search_kwargs={"k":3})

# — 3. CHAIN LCEL —————
def fmt(docs):
    return "\n\n---\n\n".join(
        f"[{d.metadata.get('source','?')}, pág.{d.metadata.get('page',0)+1}]\n{d.page_content}"
        for d in docs
    )

chain = (
    {"contexto": retriever | RunnableLambda(fmt),
     "pergunta": RunnablePassthrough(),
     "nome_doc": RunnableLambda(lambda _: "documento.pdf")}
    | prompt | ChatOllama("qwen3.6:27b", temperature=0) | StrOutputParser()
)

# — 4. USAR —————
print(chain.invoke("Qual é o prazo de garantia?"))
```

RunnablePassthrough e RunnableLambda — as peças de cola

A chain RAG usa dois Runnables especiais para montar o dicionário de variáveis do prompt:

⚡ RUNNABLEPASSTHROUGH

Passa o input sem transformar. Quando a chain recebe a string da pergunta, `RunnablePassthrough()` encaminha essa mesma string para o slot `"pergunta"` do prompt — sem modificação.

Equivalente a: `lambda x: x`

🔧 RUNNABLELAMBDA

Envolve qualquer função Python em um Runnable. Usado para:

1. Formatar os docs em string (`formatar_contexto`)
2. Injetar valores constantes (`lambda _: "nome_doc"`)
3. Qualquer transformação customizada

Qualquer função `def f(x): return y` vira um Runnable composável com o pipe `|`.

RUNNABLE_ANATOMIA.PY

```
# O que acontece quando chain.invoke("Qual é o prazo?"):
#
# Input: "Qual é o prazo?"
#   ↓
# retriever.invoke("Qual é o prazo?") → [Doc1, Doc2, Doc3]
# formatar_contexto([Doc1, Doc2, Doc3]) → "[pág.8]\n...\n---\n[pág.12]\n..."
#   ↓
# contexto = "[pág.8]\n..."           ← resultado do retriever | lambda
# pergunta = "Qual é o prazo?"         ← RunnablePassthrough() – mesma string
# nome_doc = "documento.pdf"           ← RunnableLambda(lambda _: "...")
#   ↓
# prompt.format(contexto=..., pergunta=..., nome_doc=...) → mensagens
#   ↓
# llm(mensagens) → AIMessage
#   ↓
# StrOutputParser() → string final com citação de página
```

Erros comuns no pipeline RAG

ERRO	CAUSA PROVÁVEL	SOLUÇÃO
<code>ModuleNotFoundError: fitz</code>	PyMuPDF não instalado	<code>!pip install pymupdf</code> — o pacote PyPI chama-se <code>pymupdf</code> , mas o import é <code>fitz</code>
Resposta vazia ou <i>"Não encontrei"</i> para pergunta que deveria ter resposta	<code>chunk_size</code> muito pequeno ou <code>k=1</code> recuperando poucos chunks	Aumentar <code>chunk_size</code> para 1000+ ou <code>k</code> para 5; verificar se o PDF tem texto extraível (não é scanado)
LLM ainda alucina mesmo com RAG	Prompt sem instrução de grounding explícita; <code>temperature > 0</code>	Usar o prompt com <code><instrucoes></code> e <code>temperature=0</code>
<code>KeyError: "contexto"</code> na chain	Variável no prompt não corresponde à chave no dicionário da chain	Verificar que <code>{contexto}</code> , <code>{pergunta}</code> e <code>{nome_doc}</code> no template correspondem exatamente às chaves do dict da chain
Chunks com <code>page_content</code> vazio	PDF é uma imagem escaneada sem OCR	Usar OCR antes de indexar — <code>pytesseract</code> ou Adobe Acrobat. Para o CKP02, usar PDFs com texto selecionável.

Síntese da Aula 06



RAG = memória externa sob demanda. O LLM não alucina sobre documentos que nunca viu — porque o RAG recupera o trecho relevante e injeta no contexto antes de gerar.



6 etapas: load (PyMuPDF) → split (RecursiveCharacterTextSplitter) → embed (nomic) → store (ChromaDB) → retrieve (top-k) → generate (LLM com grounding).



Chain LCEL: {contexto: retriever | lambda, pergunta: passthrough} | prompt | llm | parser — o pipeline inteiro em ~10 linhas de código declarativo.



Grounding: o prompt instrui o modelo a citar fonte e página, e a responder "não encontrei" quando a informação não está no contexto — eliminando alucinação.



SOON

Próximo: Aula 07 vai medir a qualidade desse pipeline com RAGAS (faithfulness e answer relevancy) e otimizar chunking. O RAG que você construiu hoje é o baseline.

Perguntas frequentes

PERGUNTA	RESPOSTA
"Por que <code>temperature=0</code> no RAG?"	RAG é um caso de recuperação de fatos — não queremos criatividade, queremos fidelidade ao documento. <code>Temperature=0</code> maximiza o greedy decoding: o modelo sempre escolhe o token mais provável, minimizando variações. Para chatbots criativos (Módulo 1) usamos 0.7; para RAG factual, 0 ou 0.1.
"O que é <code>k=3</code> no retriever? Devo aumentar?"	<code>k=3</code> significa recuperar os 3 chunks mais similares. Mais <code>k</code> = mais contexto, mas pode introduzir chunks menos relevantes (ruído) e usa mais tokens. Comece com <code>k=3</code> , aumente para <code>k=5</code> se as respostas parecerem incompletas. Na Aula 07 o RAGAS vai medir empiricamente o impacto de cada valor.
"E se o PDF for escaneado (imagem)?"	PyMuPDF só extrai texto de PDFs com camada de texto. Para PDFs escaneados, precisa de OCR. Para o CKP02, use PDFs com texto selecionável (você consegue copiar texto do PDF). Dica: PDFs gerados por Word, LaTeX ou exportados de browsers costumam ter texto. PDFs escaneados por câmera ou impressora precisam de OCR.
"A chain RAG tem memória entre perguntas?"	Não — a chain RAG desta aula é stateless (sem memória). Cada invoke é independente. Na Aula 08 você vai combinar RAG com <code>RunnableWithMessageHistory</code> para ter um RAG com memória de conversa — o chatbot vai lembrar da pergunta anterior ao responder.

Python novo desta aula

CONCEITOS_PYTHON_AULA06_2SEM.PY

```
# 1. extend() – adicionar todos os itens de uma lista em outra
lista_a = [1, 2]
lista_a.extend([3, 4]) # [1, 2, 3, 4] – diferente de append([3, 4]) = [1, 2, [3, 4]]

# 2. enumerate() – iterar com índice
for i, doc in enumerate(docs, 1): # começa em 1 – i=1,2,3...
    print(f"Trecho {i}: {doc.page_content}")

# 3. .get() em dict com valor padrão
pg = doc.metadata.get("page", "?") # retorna "?" se "page" não existir

# 4. join() com separador multilinha
separador = "\n\n---\n\n"
texto     = separador.join(["Trecho A", "Trecho B"])

# 5. RunnablePassthrough e RunnableLambda – LCEL avançado
from langchain_core.runnables import RunnablePassthrough, RunnableLambda

passthrough = RunnablePassthrough() # x → x (identidade)
constante   = RunnableLambda(lambda _: "doc.pdf") # ignora input, retorna constante
transformar = RunnableLambda(minha_função) # qualquer função vira Runnable

# 6. statistics.mean() – média de uma lista
import statistics
media = statistics.mean([100, 200, 150]) # 150.0
```

RAG em produção — onde o padrão aparece

SETOR FINANCEIRO E JURÍDICO

Assistente de compliance

RAG sobre milhares de páginas de regulamentos e normas internas. Analistas perguntam sobre regras específicas e recebem a resposta com citação exata do parágrafo da norma — o mesmo padrão de grounding construído nesta aula.

Assistentes de código

Ferramentas de IA para programação indexam o repositório do usuário em embeddings. Ao perguntar "como essa função é usada?", recuperam os arquivos relevantes e respondem com base no código real do projeto — não em conhecimento genérico sobre a linguagem.

Suporte ao cliente

Base de conhecimento indexada como embeddings. Quando um cliente abre um chamado, o sistema recupera os artigos de suporte mais relevantes e sugere a resolução ao atendente — com link para a fonte.

PADRÃO DE MERCADO

Praticamente todo produto de IA empresarial lançado em 2024–2026 tem RAG por baixo. O diferencial não é ter RAG — é ter RAG com boa qualidade de chunking, retrieval preciso e grounding robusto. Isso é o que a Aula 07 (RAGAS) vai ensinar a medir e otimizar.

O que vem na Aula 07 — RAG avançado e RAGAS

O QUE VOCÊ VAI APRENDER

Chunking estratégico: como tamanho e overlap afetam a qualidade

Reranking: reordenar os chunks recuperados por relevância antes de gerar

RAGAS: medir a qualidade do RAG com duas métricas objetivas:

- *faithfulness* — a resposta está ancorada no contexto?
- *answer relevancy* — a resposta responde à pergunta?

Entrega CKP02 ao final da Aula 07.

O QUE GUARDAR PARA A AULA 07

- Notebook da Aula 06 com pipeline RAG funcionando
- Os mesmos 3+ PDFs do domínio (para comparar métricas)
- 5 pares de (pergunta, resposta esperada) do domínio — serão o dataset de avaliação do RAGAS

A entrega CKP02 vai incluir as métricas RAGAS documentadas no notebook.

 **Ação antes da Aula 07:** prepare 5 pares (pergunta, resposta-esperada) sobre os documentos do domínio. Ex: ("Qual é o prazo de entrega?", "30 dias conforme cláusula 7.3"). Esses pares são o dataset de avaliação do RAGAS.

04

Demo ao vivo

RAG sobre 3 PDFs reais. A mesma pergunta, o mesmo modelo — sem RAG alucina, com RAG cita a fonte exata.



Demo Prática ao Vivo

O professor roda a mesma pergunta sobre um manual técnico antes e depois de configurar o RAG. Sem RAG, o modelo preenche a lacuna com uma resposta plausível mas inventada. Com RAG, a mesma pergunta retorna a cláusula exata, com página citada.

1. SEM RAG

Pergunta: **"Qual o prazo de garantia do produto X?"** — o modelo puro responde "12 meses conforme a legislação brasileira", um número plausível mas inventado, sem qualquer relação com o manual real.

2. INDEXAÇÃO AO VIVO

3 PDFs (manual técnico, regulamento, FAQ) são carregados, divididos em chunks e indexados no ChromaDB — o pipeline completo roda em ~2 minutos diante da turma.

3. COM RAG

A mesma pergunta agora retorna: **"Conforme o manual, página 15, o prazo de garantia é de 24 meses, cláusula 8.2"** — resposta real, extraída do documento, com fonte citada.

🛠️ STACK DA DEMO

Google Colab · Ollama Cloud API · PyMuPDF · nomic-embed-text · ChromaDB · qwen3.6:27b

Pipeline RAG completo sobre os PDFs do grupo ★★

Grupo 3-4 · 20 minutos · Google Colab · PDFs do domínio obrigatórios

1. **Complete as 4 lacunas** — caminhos dos PDFs, parâmetros do splitter, criação do vector store, montagem e invocação da chain.
2. **Teste com 3 perguntas reais** do domínio — perguntas que teriam respostas concretas nos documentos.
3. **Compare sem vs. com RAG:** faça a mesma pergunta ao ChatOllama direto e à chain_rag. Documente a diferença.
4. **Verifique o grounding:** faça uma pergunta que NÃO está nos documentos. A chain deve responder "Não encontrei essa informação" — não alucinar.

Gabarito das lacunas

Lacuna 1: os caminhos dos arquivos que acabaram de ser enviados pelo upload, usados para carregar cada PDF no loader.

Lacuna 2: o tamanho de cada chunk e a sobreposição entre eles — os mesmos valores usados na aula (800 e 100) — aplicados sobre a lista de páginas carregadas.

Lacuna 3: o nome do modelo de embedding do semestre, e os chunks e embeddings passados ao vector store na criação da coleção.

Lacuna 4: a pergunta repassada sem transformação para o prompt, e a pergunta real do domínio ao invocar a chain.



Desafio: adicionar `similarity_search_with_score` para mostrar o score de cada chunk recuperado antes de gerar a resposta. Isso vira insumo para o RAGAS da Aula 07.

Referências

PAPER

Lewis, P. et al. — "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS, 2020. O paper original que cunhou o termo RAG. arxiv.org/abs/2005.11401

DOCS

LangChain — RAG tutorial completo com PyMuPDF, Chroma e LCEL. python.langchain.com/docs/tutorials/rag

DOCS

PyMuPDF — Documentação do loader LangChain com PyMuPDF. python.langchain.com/docs/integrations/document_loaders/pymupdf

DOCS

RecursiveCharacterTextSplitter — Estratégias de chunking, parâmetros e separadores. python.langchain.com/docs/how_to/recursive_text_splitter

LIVRO

Goodfellow, I.; Bengio, Y.; Courville, A. — Deep Learning. Pearson, 2017. Cap. 15 — Representações distribuídas: a base teórica dos embeddings usados no RAG.

RAG funcional em 50 linhas

O pipeline está rodando. O modelo não alucina sobre os documentos do domínio — cita a fonte e a página. Agora falta medir se a qualidade é boa o suficiente: isso é a Aula 07.

→ **PRÓXIMA AULA — AULA 07 · 21/09**

RAG avançado — chunking estratégico, reranking e RAGAS

Medir faithfulness e answer relevancy. Otimizar o pipeline. Entregar CKP02.

Preparar para a Aula 07

5 pares (pergunta, resposta esperada) do domínio. Notebook de hoje salvo e funcionando.